

# Yuoek's Notebooks

Yuoek

2026-06-01

六月 June

Size Every Opportunity To Grow.

遇光见影，遇你见爱！

2026 年 6 月 5 日

# 目录

|                            |           |
|----------------------------|-----------|
| <b>前言</b>                  | <b>5</b>  |
| <b>LIFE</b>                | <b>7</b>  |
| <b>1 Todo</b>              | <b>9</b>  |
| <b>2 Diary</b>             | <b>13</b> |
| 2.1 2026.5. . . . .        | 15        |
| 2.2 2026.6. . . . .        | 17        |
| <b>3 Class</b>             | <b>20</b> |
| 3.1 compiler . . . . .     | 22        |
| 3.2 FIRST FOLLOW . . . . . | 24        |
| <b>FILM</b>                | <b>33</b> |
| <b>4 Words</b>             | <b>35</b> |
| 4.1 words day . . . . .    | 37        |
| <b>5 Quote</b>             | <b>40</b> |
| 5.1 quote day . . . . .    | 42        |
| <b>6 Paint</b>             | <b>55</b> |
| 6.1 paint day . . . . .    | 57        |
| 6.1.1 tikz . . . . .       | 59        |
| 6.1.2 polar axes . . . . . | 60        |
| <b>7 Poetry</b>            | <b>62</b> |
| 7.1 poetry day . . . . .   | 64        |
| <b>8 Music</b>             | <b>68</b> |
| 8.1 music day . . . . .    | 70        |
| <b>9 Film</b>              | <b>73</b> |
| 9.1 film day . . . . .     | 75        |
| <b>SCIENCE</b>             | <b>78</b> |
| <b>10 CS</b>               | <b>80</b> |
| 10.1 cs day . . . . .      | 82        |

|                            |            |
|----------------------------|------------|
| 目录                         | 4          |
| 10.2 code . . . . .        | 90         |
| 10.2.1 c . . . . .         | 92         |
| <b>11 Math</b>             | <b>105</b> |
| 11.1 math day . . . . .    | 107        |
| 11.2 cum . . . . .         | 110        |
| 11.3 symbols . . . . .     | 112        |
| <b>12 Physics</b>          | <b>115</b> |
| 12.1 physics day . . . . . | 117        |
| 12.2 draft . . . . .       | 119        |
| <b>结语</b>                  | <b>125</b> |

# 前言

Life

生活

生活不是电影，生活要艰难得多

2026 年 6 月 5 日

# LIFE

TODO

每天一个 flag

每天进步一点点

2026 年 6 月 5 日



# 1 Todo

## Notes

|                                                                                                             |     |
|-------------------------------------------------------------------------------------------------------------|-----|
| ■ 注, 如果最左推导和最右推导画出来两个语法树不同, 则有二义性, 相同则无二义性, 如上图所示。 . . . . .                                                | 23  |
| ■ 产生式有两条选择: 一个生成 $\beta$ , 一个生成 $\alpha$ , $\alpha$ 不断进行循环, 生成无数个 $\alpha$ , 因此最终生成 $\beta\alpha$ . . . . . | 23  |
| ■ The following section needs to be rewritten! . . . . .                                                    | 93  |
| ■ 引用头文件 . . . . .                                                                                           | 119 |
| ■ printf 输出函数 . . . . .                                                                                     | 119 |
| ■ 注意格式化字符串要和数据匹配 . . . . .                                                                                  | 119 |
| ■ The original todo note with changed colours. Here's another line. . . . .                                 | 119 |
| ■ Is this correct? . . . . .                                                                                | 119 |
| ■ I'm unsure about also! . . . . .                                                                          | 119 |
| ■ Change this! . . . . .                                                                                    | 120 |
| ■ This can help me in chapter seven! . . . . .                                                              | 120 |
| ■ This really needs to be improved!What was I thinking?! . . . . .                                          | 120 |
| ■ The following section needs to be rewritten! . . . . .                                                    | 120 |

Todo Day

To Find Sophiw

我在未来等你；现在去，跑着去

2026 年 6 月 5 日

# TODO

(a) 内容 1

(b) 内容 2

1. 一级

1. 二级 1.1

# Diary

## 日记

厚德载物，有容乃大

2026 年 6 月 5 日

## 2 Diary

2026

日月如梭

为虞十载，我心万宁

2026 年 6 月 5 日

**2.1 2026.5.**

## 2026 年 05 月 24 日 星期日 微风

今日专注优化 LaTeX 所有目录文档，补全 HTML、CSS 前端知识体系。修正特殊字符转义报错，统一全文排版缩进与层级结构，让笔记规整易读。

对比学习 transition 过渡与 @keyframes 关键帧动画，手写实操案例，理清两种动效原理与适用场景，做到理论和代码实践相互印证。

学习心态沉稳平和，每日稳步积累知识点，查漏补缺巩固所学内容，按照计划循序渐进提升自身能力。



**2.2 2026.6.**

## 2026 年 06 月 01 日 星期一 晴

昨天晚上心血来潮，看了 Sone ZVE10 的一些视频，感觉别人拍摄得很好，了解了 S-Log 和 HLG，了解了摄影的参数，以及怎么才能让它不过曝和欠曝。所以昨天到凌晨 1 点多才睡觉，今天还是和平时一样 5 点半醒，看了时间太困了又睡着了，6 点 10 多分才起。看了电影《桥》、《夜审》还有《油炸绿番茄》，中午看的时候真的很困，即使已经睡了一个小时左右。不知是不是整天待在寝室，总是感觉晚上很虚弱无力。在 LaTeX 做了一些插入视频、音频还有图片的测试，之前已经可以添加笔记编注和代码。还有一点就是每次打开 text 文件，进行实时编译，再启动 VNC，会发现平板特别卡，原因是启动了 LaTeX 编译后台进程占用很多资源空间。

Class

课程

千教万教教人求真

2026 年 6 月 5 日

### **3 Class**

compiler

编译原理

为虞十载，我心万宁

2026 年 6 月 5 日

### **3.1 compiler**

## 编译原理笔记

### 语言的文法

#### 短语到句柄

##### 语法树最左推导最右推导短语直接短语素短语句柄

最左推导: 见左拆左。

最右推导: 见右拆右。

句型: 一棵语法树对应的整串符号。

短语: 任意一棵子树的全部叶子节点串。

直接短语: 子树只有父子两层 (层深度只有 2), 并且子层为叶子节点。

素短语: 子树至少包含一个终结符, 并且只有一个短语, 其内部不再包含更小的素短语。

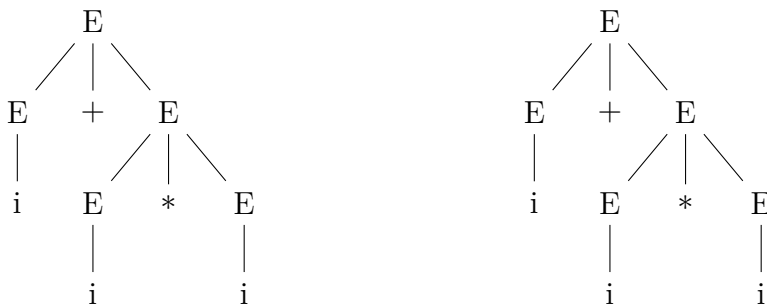
句柄: 句型最左边那个直接短语。

例:

文法  $G[E]: E \rightarrow E + E \mid E * E \mid (E) \mid i$ , 字符串:  $i+i*i$

最左推导:  $E \rightarrow E + E \rightarrow i + E \rightarrow i + E * E \rightarrow i + i * E \rightarrow i + i * i$

最右推导:  $E \rightarrow E + E \rightarrow E + E * E \rightarrow E + E * i \rightarrow E + i * i \rightarrow i + i * i$



注: 如果最左推导和最右推导画出来两个语法树不同, 则有二义性, 相同则无二义性, 如上图所示。

短语:  $i+i*i$ ,  $i*i$ ,  $i$

直接短语:  $i$

素短语:  $i$

句柄:  $i$

#### 消除左递归

$$A \rightarrow A\alpha \mid \beta \quad \text{化为} \quad \begin{cases} A \rightarrow \beta A' \\ A' \rightarrow \alpha A' \mid \varepsilon \end{cases} \quad (\alpha, \beta \text{ 为任意字符串})$$

产生式有两条选择: 一个生成  $\beta$ , 一个生成  $\alpha$ ,  $\alpha$  不断进行循环, 生成无数个  $\alpha$ , 因此最终生成  $\beta\alpha$

$$\text{例 1: } G(E) \quad \begin{cases} E \rightarrow E + T \mid T \\ T \rightarrow T * F \mid F \\ F \rightarrow (E) \mid i \end{cases} \Rightarrow \begin{cases} E \rightarrow TE' \\ E' \rightarrow +TE' \mid \varepsilon \\ T \rightarrow FT' \\ T' \rightarrow *FT' \mid \varepsilon \\ F \rightarrow (E) \mid i \end{cases}$$

### 提取公因式

$$A \rightarrow \delta\beta_1 \mid \delta\beta_2 \mid \cdots \mid \delta\beta_i \mid \gamma_1 \mid \gamma_2 \mid \cdots \mid \gamma_j \quad \text{化为} \quad \begin{cases} A \rightarrow \delta A' \mid \gamma_1 \mid \gamma_2 \mid \cdots \mid \gamma_j \\ A' \rightarrow \beta_1 \mid \beta_2 \mid \cdots \mid \beta_i \end{cases}$$

例 1: 已知文法

$$G(A) : \begin{cases} A \rightarrow aAB \mid a \\ B \rightarrow Bb \mid d \end{cases} \quad \Downarrow$$

$$A \rightarrow aA' \quad A' \rightarrow AB \mid \varepsilon$$

$$B \rightarrow dB' \quad B' \rightarrow bB' \mid \varepsilon$$

例 2:

$$G[A] : A \rightarrow aAB \mid a \mid b \quad \Downarrow$$

$$A \rightarrow aA' \mid b$$

$$A' \rightarrow AB \mid \varepsilon$$

## 3.2 FIRST FOLLOW

First 集: 看产生式左边。

Follow 集: 看产生式右边。

- $$\left\{ \begin{array}{l} \text{以 } \# \text{ 开头.} \\ \text{若非终结符后面为空, 则将推导式左边加入 Follow 集.} \\ \text{若非终结符后面不为空, 如果不为空为终结符, 直接将终结符加入, 如果不为空为非终结符, 则将} \end{array} \right.$$



**例 1:**

$$G[S] : \begin{cases} S \rightarrow aH \\ H \rightarrow aMd \mid d \\ M \rightarrow Ab \mid \varepsilon \\ A \rightarrow aM \mid e \end{cases} \quad \begin{array}{l} \text{First 集} \\ \text{First}(S) = \{a\} \\ \text{First}(H) = \{a, d\} \\ \text{First}(M) = \{\varepsilon\} \cup \text{First}(A) = \{a, e, \varepsilon\} \\ \text{First}(A) = \{a, e\} \end{array}$$

**例 2:**

$$G(E) : \begin{cases} E \rightarrow TE' & \text{First}(E) = \{ (, a, b, \Lambda \} \\ E' \rightarrow +E \mid \varepsilon & \text{First}(E') = \{ +, \varepsilon \} \\ T \rightarrow FT' & \text{First}(T) = \{ (, a, b, \Lambda \} \\ T' \rightarrow T \mid \varepsilon & \text{First}(T') = \{ (, a, b, \Lambda, \varepsilon \} \\ F \rightarrow PF' & \text{First}(F) = \{ (, a, b, \Lambda \} \\ F' \rightarrow *F' \mid \varepsilon & \text{First}(F') = \{ *, \varepsilon \} \\ P \rightarrow (E) \mid a \mid b \mid \Lambda & \text{First}(P) = \{ (, a, b, \Lambda \} \end{cases}$$

**例 3:**

求例 1 中文法的 Follow 集

$$\text{Follow}(S) = \{ \# \}$$

$$\text{Follow}(H) = \{ \# \}$$

$$\text{Follow}(M) = \{ d \} \cup \text{Follow}(A) = \{ d, b \}$$

$$\text{Follow}(A) = \{ b \}$$

$$\text{例 4: 求例 2 中文法的 Follow 集} \quad \begin{cases} E \rightarrow \varepsilon \\ T' \rightarrow \varepsilon \\ F' \rightarrow \varepsilon \end{cases}$$

$$\text{Follow}(E) = \{ \#, ) \}$$

$$\text{Follow}(E') = \{ \#, ) \}$$

$$\text{Follow}(T) = \{ +, \#, ) \}$$

$$\text{Follow}(T') = \{ +, \#, ) \}$$

$$\text{Follow}(F) = \{ (, a, b, \Lambda, +, \#, ) \}$$

$$\text{Follow}(F') = \{ (, a, b, \Lambda, +, \#, ) \}$$

$$\text{Follow}(P) = \{ *, (, a, b, +, \Lambda, \#, ) \}$$

## DFA 转换

### LL(1)

消除左递归提取公因式

求 FIRST FOLLOW 集

画分析表 (行非终结, 列终结, 空跟 FOLLOW, 不空跟 FIRST)

判断 LL(1) (不空取 First, 含空要并 Follow, 同左 Select 不交, 就是 LL(1))

分析过程 (符号栈输入符号剩余符号, 符号栈非终结替换, 右部反过来写)

$$\text{例 1: 已知文法 } G[A] : \begin{cases} A \rightarrow aAB \mid a \\ B \rightarrow Bb \mid d \end{cases}$$

求  $G[A]$  等价的 LL(1) 文法  $G'[A]$

过程: 先消除左递归和回溯, 得到改造后的文法  $G'(A)$

$$A \rightarrow aA'$$

$$A' \rightarrow AB \mid \varepsilon$$

$$B \rightarrow dB'$$

$$B' \rightarrow bB' \mid \varepsilon$$

再求 First 集和 Follow 集

$$\text{First}(A) = \{a\} \quad \text{Follow}(A) = \{\#, d\}$$

$$\text{First}(A') = \{a, \varepsilon\} \quad \text{Follow}(A') = \{\#, d\}$$

$$\text{First}(B) = \{d\} \quad \text{Follow}(B) = \{\mid\}$$

$$\text{First}(B') = \{b, \varepsilon\} \quad \text{Follow}(B') = \{\mid\}$$

最后构造  $G'[A]$  的 LL(1) 分析表 行: 非终结符, 列: 终结符

|      | $a$                      | $b$                       | $d$                          | $\#$                         |
|------|--------------------------|---------------------------|------------------------------|------------------------------|
| $A$  | $A \rightarrow aA'$      |                           |                              |                              |
| $A'$ | $A' \rightarrow AB \mid$ |                           | $A' \rightarrow \varepsilon$ | $A' \rightarrow \varepsilon$ |
| $B$  |                          |                           | $B \rightarrow dB'$          |                              |
| $B'$ |                          | $B' \rightarrow bB' \mid$ | $B' \rightarrow \varepsilon$ |                              |

对于推导式  $X \rightarrow ?$ , 若推导式右侧结果为空, 则去 Follow 集中找到  $\text{Follow}(X)$

表达式填入行为  $X$ 、列为

$\text{Follow}(X)$  中元素的位置, 如例 1 中  $A' \rightarrow \varepsilon$ ,  $\text{Follow}(A') = \{\#, d\}$ , 则在  $(A', d)$   $(A', \#)$  中填写  $A' \rightarrow \varepsilon$

若 $X \rightarrow ?$  推导式右侧结果不为空, 则在行为 $X$ , 列为 $\text{First}(?)$  处填写表达式  
 $X \rightarrow ?$ , 如 $A \rightarrow aA'$ , 在 $(A, a)$ 处填写 $A \rightarrow aA'$

### 例 2: 将下列文法改造为 LL(1) 文法

$$G[V] : \begin{cases} V \rightarrow N \mid N[E] \\ E \rightarrow V \mid V + E \\ N \rightarrow i \end{cases}$$

消除回溯

$$V \rightarrow NV'$$

$$V' \rightarrow \varepsilon \mid [E]$$

$$E \rightarrow VE'$$

$$E' \rightarrow \varepsilon \mid +E$$

$$N \rightarrow i$$

求 First 集、Follow 集

$$\begin{array}{ll} \text{First}(V) = \{i\} & \text{Follow}(V) = \{\#, +\} \\ \text{First}(V') = \{\varepsilon, [\} & \text{Follow}(V') = \{\#, +\} \\ \text{First}(E) = \{i\} & \text{Follow}(E) = \{]\} \\ \text{First}(E') = \{\varepsilon, +\} & \text{Follow}(E') = \{]\} \\ \text{First}(N) = \{i\} & \text{Follow}(N) = \{[, \#, +\} \end{array}$$

对于

$$V' \rightarrow \varepsilon \mid [E]$$

$$\text{First}(\varepsilon) \cap \text{First}([\} = \emptyset$$

$$\text{Follow}(V') \cap \text{First}([\} = \{\#, +\} \cap \{[\} = \emptyset$$

对于

$$E' \rightarrow \varepsilon \mid +E$$

$$\text{First}(\varepsilon) \cap \text{First}(+) = \emptyset$$

$$\text{Follow}(E') \cap \text{First}(+) = \{+\} \cap \{]\} = \emptyset$$

$G'[V]$ 符合 LL(1) 文法

对于

$$V' \rightarrow \varepsilon \mid [E]$$

$$\text{First}(\varepsilon) \cap \text{First}([\ ] ) = \emptyset$$

$$\text{Follow}(V') \cap \text{First}([\ ] ) = \{\#, +\} \cap \{[\ ]\} = \emptyset$$

对于

$$E' \rightarrow \varepsilon \mid +E$$

$$\text{First}(\varepsilon) \cap \text{First}(+) = \emptyset$$

$$\text{Follow}(E') \cap \text{First}(+) = \{+\} \cap \{[\ ]\} = \emptyset$$

$G'[V]$ 符合 LL(1) 文法

**例 3:** 对于例 1 中的文法, 给出输入串  $aad \mid \#$  分析过程

推导式 (反着压)

| 符号栈         | 当前输入符号 | 输入串           |
|-------------|--------|---------------|
| $\#A$       | $a$    | $aad \mid \#$ |
| $\#A'a$     | $a$    | $aad \mid \#$ |
| $\#A'$      | $a$    | $d \mid \#$   |
| $\mid BA$   | $a$    | $d \mid \#$   |
| $\mid BA'a$ | $a$    | $d \mid \#$   |
| $\mid BA'$  | $d$    | $\mid \#$     |
| $\mid B$    | $d$    | $\mid \#$     |
| $\mid B'd$  | $d$    | $\mid \#$     |
| $\mid B'$   | $\mid$ | $\#$          |
| $\mid$      | $\mid$ | $\#$          |
| $\#$        | $\#$   |               |

**求 select 集:**

$\text{select}(A \rightarrow \alpha)$

若 $\alpha$ 推不出 $\varepsilon$ , 则 $\text{select}(A \rightarrow \alpha) = \text{First}(\alpha)$

若 $\alpha$ 可推出 $\varepsilon$ , 则 $\text{select}(A \rightarrow \alpha) = (\text{First}(\alpha) - \{\varepsilon\}) \cup \text{Follow}(A)$

可以修正例 1 中:

$\text{select}(A' \rightarrow AB \mid) = \text{First}(A) = \{a\}$

$\text{select}(A' \rightarrow \varepsilon) = \{\#, d\}$

$\text{select}(A' \rightarrow AB \mid) \cap \text{select}(A' \rightarrow \varepsilon) = \emptyset$

$\text{select}(B' \rightarrow bB') = \{b\}$

$\text{select}(B' \rightarrow \varepsilon) = \{\mid\}$

$\text{select}(B' \rightarrow bB') \cap \text{select}(B' \rightarrow \varepsilon) = \emptyset$

进而判断 LL(1) 文法的另一个等价条件为:

对任意一个

若存在的 $A \rightarrow \alpha \mid \beta$  类表达式,

且要满足 $\text{select}(A \rightarrow \alpha) \cap \text{select}(A \rightarrow \beta) = \emptyset$

## LR(0)

I0 添加小圆点

小圆点后跟非终结符展开

小圆点在最后面, 归约

SLR 归约要 FOLLOW

判断 SLR 看移进符是否在归约集, 不在是 SLR, 在不是

LR(1) 引用展望符

LALR 合并展望符

## 构造 LR(0) 分析表

拓广文法  $\rightarrow$  列项目  $\rightarrow$  项目集规范簇  $\rightarrow$  分析表

**例 1: 构造**  $G[E]: \begin{cases} E \rightarrow E + T \mid T \\ T \rightarrow (E) \mid a \end{cases}$  的 LR(0) 分析表

拓广文法  $G[E']:$

(0)  $E' \rightarrow E$

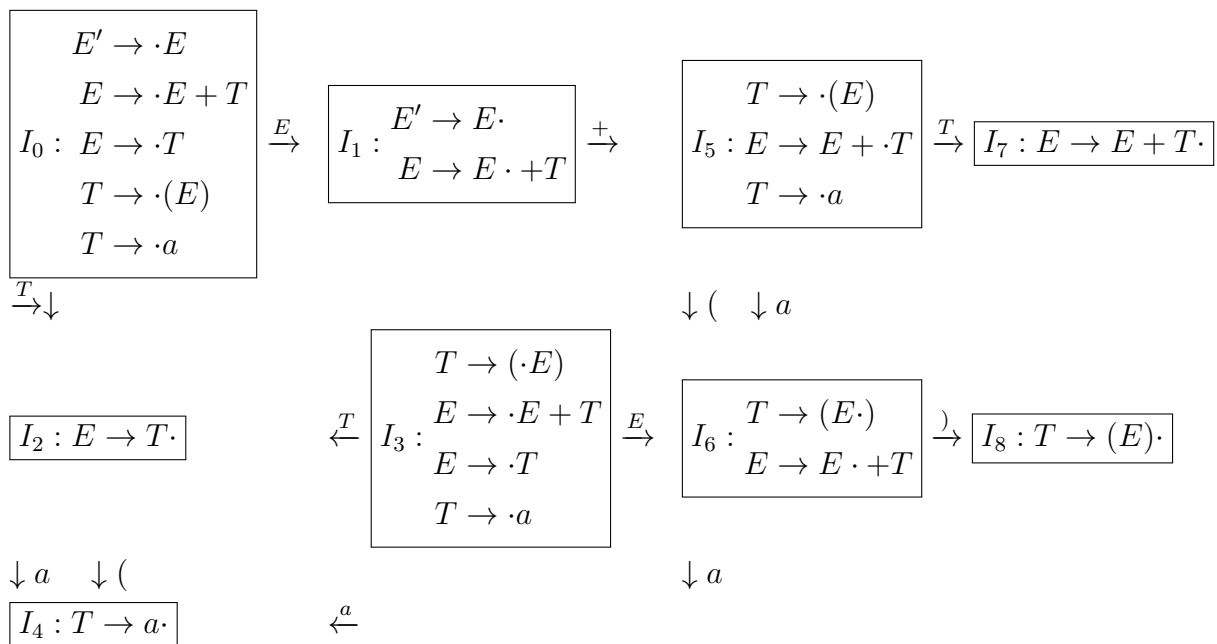
(1)  $E \rightarrow E + T$

(2)  $E \rightarrow T$

(3)  $T \rightarrow (E)$

(4)  $T \rightarrow a$

所有点位于推导式右侧首位



点后为非终结符,

除了拓广文法的第一项,

均加入集合中

此时遇到的非终结符  $E$ ,

由  $E \rightarrow T$ , 又得到一个  $T$

再求 Action-Goto 表

| 状态 | Action(终结符)           |                       |                       |                       |                       | GOTO(非终结符) |          |
|----|-----------------------|-----------------------|-----------------------|-----------------------|-----------------------|------------|----------|
|    | <i>a</i>              | +                     | (                     | )                     | #                     | <i>E</i>   | <i>T</i> |
| 0  | <i>S</i> <sub>4</sub> |                       |                       | <i>S</i> <sub>3</sub> |                       | 1          | 2        |
| 1  |                       | <i>S</i> <sub>5</sub> |                       |                       | acc                   |            |          |
| 2  | <i>r</i> <sub>2</sub> | <i>r</i> <sub>2</sub> | <i>r</i> <sub>2</sub> | <i>r</i> <sub>2</sub> | <i>r</i> <sub>2</sub> |            |          |
| 3  | <i>S</i> <sub>4</sub> |                       |                       | <i>S</i> <sub>3</sub> |                       | 6          | 2        |
| 4  | <i>r</i> <sub>4</sub> | <i>r</i> <sub>4</sub> | <i>r</i> <sub>4</sub> | <i>r</i> <sub>4</sub> | <i>r</i> <sub>4</sub> |            |          |
| 5  | <i>S</i> <sub>4</sub> |                       |                       | <i>S</i> <sub>3</sub> |                       | 7          |          |
| 6  |                       | <i>S</i> <sub>5</sub> |                       |                       | <i>S</i> <sub>8</sub> |            |          |
| 7  | <i>r</i> <sub>1</sub> | <i>r</i> <sub>1</sub> | <i>r</i> <sub>1</sub> | <i>r</i> <sub>1</sub> | <i>r</i> <sub>1</sub> |            |          |
| 8  | <i>r</i> <sub>3</sub> | <i>r</i> <sub>3</sub> | <i>r</i> <sub>3</sub> | <i>r</i> <sub>3</sub> | <i>r</i> <sub>3</sub> |            |          |

点在最后，且找不到可通往其它状态，根据推导式填 $r_i$   
状态 $\alpha$ 经 Action 到状态 $\beta$  则在 $(\alpha, a_i)$ 处填写 $S_\beta$

$a_i$

经 GOTO  $X$ 到状态 $\gamma$  则在 $(\alpha, X)$ 处填 $\gamma$   
若状态 $\delta$ 下，含有拓广文法中  $(0)$ ，且为 $X' \rightarrow X \cdot$ 形式，  
则在 $(\delta, \#)$ 处填写 acc

SLR(1)

LR(1)

语法制导的翻译与属性文法

S/L 属性文法与翻译模式

源程序的中间表示

四元式

算符优先

波兰表达式

FILM

你与梦想

电影让一切梦想具体化

2026 年 6 月 5 日



## FILM

# WORDS

## 字词汇集

一沙一世界，一字一语言

2026 年 6 月 5 日

## 4 Words

# Words Day

## 句读

咿呀学语，见字如面

2026 年 6 月 5 日

## 4.1 words day

## TODO

- 第一项
- 第二项
- 第三项

(a) 内容 1

(b) 内容 2

1. 一级

1. 二级 1.1

TODO:

(1) 电影《黑水仙》

(2) 名人名言

(3) Latex 公式

(4) 磁：无限长

(5) 编译原理：代码优化

vi) poetry

vii) Todo

viii) Todo

ix) Todo

J) Todo

K) Todo

L) Todo

m) 内容

n) Todo

o) Todo

# QUOTE

## 一言一行

从群山中，我将为你捎来幸福的花束

2026 年 6 月 5 日

## 5 Quote



## Quote Day

千言万语

让我在你的沉默中安静无声

2026 年 6 月 5 日

## 5.1 quote day

这是每日一句。

**2026-05-26**

Our patience will achieve more than our force.

我们的耐心将比武力取得更多成就。

— Edmund Burke(埃德蒙·伯克)

**2026-05-27**

Shoot for the moon. Even if you miss, you'll land among the stars.

瞄准月亮吧，即使你没射中，也会落在星辰之间。

— Oscar Wilde(奥斯卡·王尔德)

**2026-05-28**

Courage does not always roar. Sometimes courage is the quiet voice at the end of the day saying, 'I will try again tomorrow.'

勇气不总是咆哮。有时候，勇气是一天结束时那安静低语：“明天我会再试一次”。

— Mary Anne Radmacher(玛丽·安妮·拉德马赫)

**2026-05-29**

The secret to eternal youth is arrested development.

永葆青春的秘诀是停滞的发育。

— Alice Roosevelt Longworth(爱丽丝·罗斯福·朗沃思)

**2026-05-30**

It really doesn't matter if the person who hurt you deserves to be forgiven. Forgiveness is a gift you give yourself. You have things to do and you want to move on.

伤害你的人是否值得被原谅，其实并不重要。宽恕是你送给自己的礼物。你有事情要做，你想继续前行。

— 戈登·阿特金森

**2026-05-31**

To achieve the impossible dream, try going to sleep.

要实现不可能的梦想，试试去睡觉吧。

— 琼·克莱普纳



**2026-06-01**

It's never too late to enjoy happy childhood.

享受一个快乐的童年，永远不嫌晚。

— Tom Robbins(汤姆·罗宾斯)

**2026-06-02**

Our progress as a nation can be no swifter than our progress in education. The human mind is our fundamental resource.

我们国家的进步速度，不可能快于我们在教育上的进步。人的头脑是我们最基本的资源。

— John F · Kennedy(约翰 · F · 肯尼迪)

**2026-06-03**

Without music, life would be a mistake.

没有音乐，生命将是一种错误。

—

**2026-06-04**

Nobody can be exactly like me. Sometimes even I have trouble doing it.

没有人能完全像我，有时候我也做不到。

—

**2026-06-05**

Be generous, be delicate, and always pursue the prize.

胸怀慷慨，心思细腻，始终瞄准目标。

— Henry James(亨利·詹姆斯)

PAINT

审美绘画

倘若不是因为你的眼睛

2026 年 6 月 5 日

## 6 Paint

# Paint Day

记忆的留白

你的颜色与形状正是我喜欢的样子

2026 年 6 月 5 日



## 6.1 paint day

## 26 May

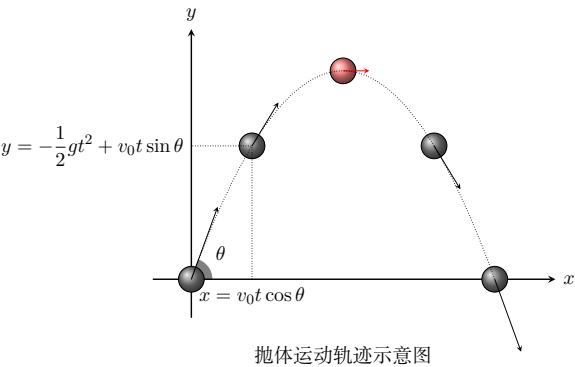
Our patience will achieve more than our force.

我们的耐心将比武力取得更多成就。

— Edmund Burke(埃德蒙·伯克)

TODO: 2026-05-26

6.1.1 tikz



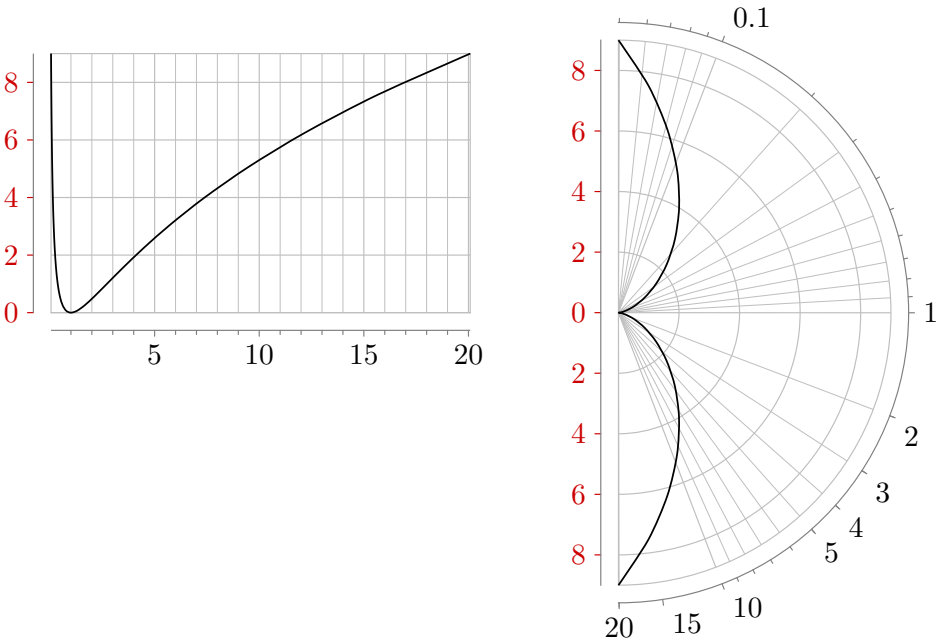
cal

|               |               |               |               |               |               |    |
|---------------|---------------|---------------|---------------|---------------|---------------|----|
| <del>11</del> | <del>12</del> | <del>13</del> | <del>14</del> | <del>15</del> | <del>16</del> | 17 |
| <del>18</del> | <del>19</del> | <del>20</del> | <del>21</del> | <del>22</del> | <del>23</del> | 24 |
| <del>25</del> | <del>26</del> | <del>27</del> | <del>28</del> | <del>29</del> | <del>30</del> | 31 |

June 2026

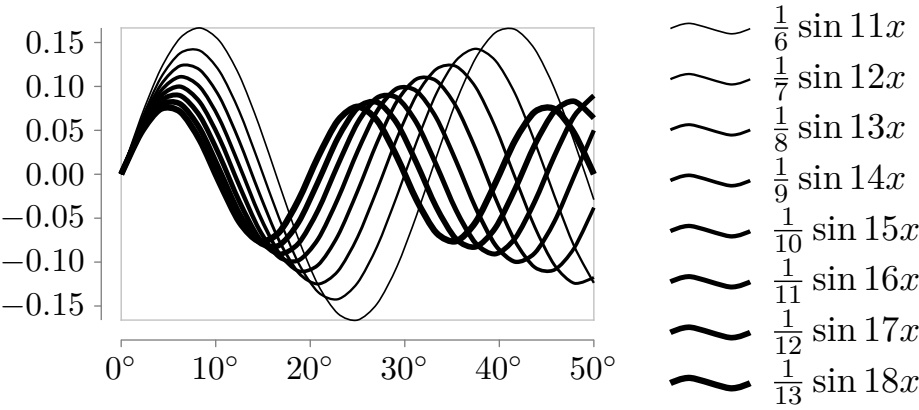
|              |              |              |              |    |    |    |
|--------------|--------------|--------------|--------------|----|----|----|
| <del>1</del> | <del>2</del> | <del>3</del> | <del>4</del> | 5  | 6  | 7  |
| 8            | 9            | 10           | 11           | 12 | 13 | 14 |
| 15           | 16           | 17           | 18           | 19 | 20 | 21 |
| 22           | 23           | 24           | 25           | 26 | 27 | 28 |
| 29           | 30           |              |              |    |    |    |

6.1.2 polar axes



tikzdatavisualizationset

vary thickness



# POETRY

## 诗歌美文

秘密地，介于阴影与灵魂之间

2026 年 6 月 5 日

## 7 Poetry

# Poetry Day

岁月如诗

是出发的时刻了，哦遭遗忘的人

2026 年 6 月 5 日

## 7.1 poetry day



## 26 May

Our patience will achieve more than our force.

我们的耐心将比武力取得更多成就。

— Edmund Burke(埃德蒙·伯克)

TODO: 2026-05-26

2026-06-01

**安静的女孩**

— 兰斯顿·休斯

倘若不是因为你的眼睛，  
我会把你比作  
没有星星的夜晚。  
倘若不是因为你的歌声，  
我会把你比作  
无梦的睡眠。

**Quite Girl**

— Langston Hughes

I would liken you  
To a night without star  
Were it not for your eyes.  
I would liken you  
To a sleep without dreams  
Were it not for your songs.

# Music

## 声音乐章

你是一只梦的蝴蝶，如歌悲鸣

2026 年 6 月 5 日

## 8 Music

# Music Day

## 精神粮食

我曾无与伦比地歌吟过你

2026 年 6 月 5 日

## 8.1 music day

## 26 May

Our patience will achieve more than our force.

我们的耐心将比武力取得更多成就。

— Edmund Burke(埃德蒙·伯克)

TODO: 2026-05-26

Film

光影视听

你与电影，爱情与梦想

2026 年 6 月 5 日



## 9 Film

# Film Day

精神粮食

电影让生命延长至少三倍

2026 年 6 月 5 日

## 9.1 film day

## 26 May

Our patience will achieve more than our force.

我们的耐心将比武力取得更多成就。

— Edmund Burke(埃德蒙·伯克)

TODO: 2026-05-26

SCIENCE

格物致知

你如同一个吻，封碱了你的嘴

2026 年 6 月 5 日

## SCIENCE

CS

计算机

程序 = 数据 + 算法

2026 年 6 月 5 日

## 10 CS



CS Day

计算机

一天一算法，10 皆世界

2026 年 6 月 5 日

**10.1 cs day**

## 26 May

Our patience will achieve more than our force.

我们的耐心将比武力取得更多成就。

— Edmund Burke(埃德蒙·伯克)

TODO: 2026-05-26

2026-06-01

## Python 贪吃蛇

pip install pygame

```
1 snake_game/  
2 --- main.py      # 程序入口  
3 --- config.py    # 全局配置(尺寸、颜色、速度)  
4 --- snake.py     # 蛇类封装  
5 --- food.py      # 食物类封装  
6 --- game.py      # 游戏逻辑主类
```

### config.py

```
1 # 窗口与格子配置  
2 WIDTH = 600  
3 HEIGHT = 400  
4 BLOCK_SIZE = 20  
5 SPEED = 12  
6  
7 # 颜色 RGB  
8 BLACK = (0, 0, 0)  
9 WHITE = (255, 255, 255)  
10 RED = (255, 0, 0)  
11 GREEN = (0, 255, 0)  
12 BLUE = (0, 0, 255)
```

### snake.py

```
1 import pygame  
2 from config import BLOCK_SIZE, GREEN  
3  
4 class Snake:  
5     def __init__(self):  
6         self.body = [(100,100), (80,100), (60,100)] # 身体坐标列表  
7         self.dir = (BLOCK_SIZE, 0) # 初始向右  
8
```

```

9     def change_dir(self, new_dir):
10         # 禁止反向掉头
11         if (new_dir[0] != -self.dir[0]) or (new_dir[1] != -self.dir[1]):
12             self.dir = new_dir
13
14     def move(self, grow=False):
15         head_x, head_y = self.body[0]
16         new_head = (head_x + self.dir[0], head_y + self.dir[1])
17         self.body.insert(0, new_head)
18         if not grow:
19             self.body.pop()
20
21     def draw(self, screen):
22         for pos in self.body:
23             rect = pygame.Rect(pos[0], pos[1], BLOCK_SIZE-1, BLOCK_SIZE-1)
24             pygame.draw.rect(screen, GREEN, rect)
25
26     def get_head(self):
27         return self.body[0]

```

### food.py

```

1  import random
2  import pygame
3  from config import BLOCK_SIZE, WIDTH, HEIGHT, RED
4
5  class Food:
6      def __init__(self):
7          self.reset_pos()
8
9      def reset_pos(self):
10         # 随机生成在格子上
11         self.x = random.randrange(0, WIDTH-BLOCK_SIZE, BLOCK_SIZE)
12         self.y = random.randrange(0, HEIGHT-BLOCK_SIZE, BLOCK_SIZE)
13         self.pos = (self.x, self.y)
14
15     def draw(self, screen):

```

```
16     rect = pygame.Rect(self.x, self.y, BLOCK_SIZE-1, BLOCK_SIZE-1)
17     pygame.draw.rect(screen, RED, rect)
```

### game.py

```
1  import pygame
2  from snake import Snake
3  from food import Food
4  from config import *
5
6  class Game:
7      def __init__(self):
8          pygame.init()
9          self.screen = pygame.display.set_mode((WIDTH, HEIGHT))
10         pygame.display.set_caption("经典贪吃蛇")
11         self.clock = pygame.time.Clock()
12         self.font = pygame.font.SysFont(None, 35)
13         self.snake = Snake()
14         self.food = Food()
15         self.score = 0
16         self.game_over = False
17
18     def check_collision(self):
19         head = self.snake.get_head()
20         # 撞墙
21         if head[0]<0 or head[0]>=WIDTH or head[1]<0 or head[1]>=HEIGHT:
22             return True
23         # 撞到自己身体
24         if head in self.snake.body[1:]:
25             return True
26         return False
27
28     def eat_food(self):
29         if self.snake.get_head() == self.food.pos:
30             self.score +=10
31             self.food.reset_pos()
32             return True
```

```
33     return False
34
35 def event_handle(self):
36     for event in pygame.event.get():
37         if event.type == pygame.QUIT:
38             pygame.quit()
39             exit()
40         if event.type == pygame.KEYDOWN:
41             if event.key == pygame.K_UP:
42                 self.snake.change_dir((0, -BLOCK_SIZE))
43             elif event.key == pygame.K_DOWN:
44                 self.snake.change_dir((0, BLOCK_SIZE))
45             elif event.key == pygame.K_LEFT:
46                 self.snake.change_dir((-BLOCK_SIZE, 0))
47             elif event.key == pygame.K_RIGHT:
48                 self.snake.change_dir((BLOCK_SIZE, 0))
49
50 def draw_ui(self):
51     text = self.font.render(f"Score:{self.score}", True, WHITE)
52     self.screen.blit(text, [0,0])
53
54 def run(self):
55     while not self.game_over:
56         self.event_handle()
57         grow = self.eat_food()
58         self.snake.move(grow)
59
60         if self.check_collision():
61             self.game_over = True
62
63         self.screen.fill(BLACK)
64         self.snake.draw(self.screen)
65         self.food.draw(self.screen)
66         self.draw_ui()
67
68         pygame.display.update()
69         self.clock.tick(SPEED)
```

```
70
71     # 游戏结束弹窗
72     over_text = self.font.render("Game Over! 关闭窗口退出", True,
73                                   WHITE)
74     self.screen.blit(over_text, [WIDTH//6, HEIGHT//2])
75     pygame.display.update()
76     while True:
77         for e in pygame.event.get():
78             if e.type == pygame.QUIT:
79                 pygame.quit()
80                 return
```

### main.py

```
1 from game import Game
2
3 if __name__ == "__main__":
4     game = Game()
5     game.run()
```



CODE

代码人生

手指敲打键盘如你敲击琴键

2026 年 6 月 5 日

**10.2 code**

Code C

代码 C

你如同一个吻，封碱了你的嘴

2026 年 6 月 5 日

**10.2.1 c**

# 第一章 C 语言入门认知

## 1.1 编程语言基础概念

- 机器语言、汇编语言、高级语言区别
- C 语言发展历史、标准版本与迭代
- C 语言核心特点：高效、跨平台、可移植
- C 语言主流应用领域：系统、嵌入式、驱动、算法

The following section needs to be rewritten!

## 1.2 程序编译运行原理

- 源代码、目标文件、可执行文件概念
- 预处理、编译、汇编、链接四步流程
- 编译报错、运行报错、逻辑报错区分

## 1.3 开发环境搭建与使用

- GCC 编译器安装与基础命令使用
- VSCode 配置 C 语言编译运行环境
- Vim 编辑器代码编写基础操作
- 首个 HelloWorld 程序逐行解析

## 1.4 C 语言基础语法框架

- 程序入口 main() 函数作用与写法
- 语句结束分号、代码块大括号规范
- 关键字、标识符、标识符命名规则
- 单行注释、多行注释 `** **` 使用场景

# 第二章基础数据与运算体系

## 2.1 基本数据类型

- 短整型 short、基本整型 int、长整型 long
- 超长整型 long long 取值范围与内存占用
- 单精度浮点 float、双精度浮点 double
- 字符类型 char、无符号字符 unsigned char
- 布尔类型 `_Bool` 逻辑真假表示

## 2.2 类型修饰限定符

- signed 有符号、unsigned 无符号修饰
- 数据类型取值范围边界理解
- sizeof 运算符计算变量、类型字节大小
- 不同类型数据自动转换、强制转换规则

## 2.3 常量分类

- 整型常量、浮点常量、字符常量
- 字符串常量存储本质与结束标志
- #define 宏定义符号常量
- const 修饰只读常量特性与用法

## 2.4 变量定义与使用

- 变量声明、定义、初始化三种形式
- 变量内存分配原理与地址概念
- 局部变量、全局变量基础区分
- 变量命名禁忌与规范书写习惯

## 2.5 运算符全集

- 算术运算符：加、减、乘、除、取模
- 自增 ++、自减--前置后置区别
- 赋值运算符与复合赋值运算
- 大于、小于、相等、不等关系运算
- 逻辑与 &&、逻辑或 ||、逻辑非! 运算
- 按位与 &、按位或 |、异或 ^ 取反
- 左移 «、右移 » 位运算实操
- 三目条件运算?:、逗号表达式运算

## 2.6 表达式与运算优先级

- 合法表达式书写规则
- 运算符优先级高低排序
- 运算符结合性：左结合、右结合
- 复杂表达式分步拆解计算思路

## 2.7 标准输入输出

- printf 格式化输出全部格式控制符
- 十进制、八进制、十六进制格式打印
- scanf 安全读取数据与地址传参
- getchar、putchar 单个字符读写
- puts、fgets 字符串读取与输出

# 第三章 程序流程控制结构

## 3.1 顺序结构

- 自上而下顺序执行逻辑
- 简单数值计算、数据交换案例

## 3.2 分支选择结构

- if 单分支条件判断语句
- if-else 双分支二选一逻辑
- if-else if-else 多分支区间判断
- switch 多路分支匹配语法规范
- switch 中 break 跳出、default 默认分支
- 分支语句多层嵌套逻辑分析

## 3.3 循环遍历结构

- while 循环：先判断条件后执行代码
- do-while 循环：先执行再判断条件
- for 循环初始化、条件、更新三段式
- 单层循环求和、计数、遍历案例
- 双层循环嵌套行列打印、矩阵处理
- 多层循环嵌套执行顺序理解

## 3.4 循环跳转语句

- break 终止当前循环与分支结构
- continue 跳过本次剩余循环内容
- goto 无条件跳转语句慎用场景
- 死循环成因、危害与合理应用

## 第四章数组与字符串处理

### 4.1 一维数组

- 一维数组定义、多种初始化方式
- 数组下标访问元素、下标越界危害
- 数组遍历、求和、平均值计算
- 数组最大值、最小值查找定位
- 数组元素查找、替换、删除操作
- 数组逆序、排序基础操作

### 4.2 二维数组

- 二维数组行列定义与初始化
- 二维数组内存连续存储规律
- 按行遍历、按列遍历两种方式
- 矩阵输入输出、矩阵加法运算
- 矩阵行列最值、对角线数据处理

### 4.3 字符数组

- 字符数组定义与单个字符存储
- 字符串结束符\0 底层原理
- 字符数组与普通数组异同点

### 4.4 字符串标准库函数

- strlen 计算字符串有效长度
- strcpy、strncpy 字符串安全拷贝
- strcat、strncat 字符串拼接合并
- strcmp、strncmp 字符串大小比较
- strchr 字符查找、strstr 子串查找

### 4.5 字符串综合实操

- 字符串大小写相互转换
- 字符串反转、空格去除处理
- 字符串分割、统计字符个数



## 第五章函数模块化编程

### 5.1 函数基础概念

- 函数作用：代码复用、简化结构
- 函数完整构成：返回值、函数名、参数
- 函数定义、函数声明、函数调用
- 头文件引用与函数声明关联

### 5.2 函数参数与返回值

- 形式参数、实际参数对应关系
- 无参函数、有参函数写法区别
- return 返回数据、终止函数执行
- void 无返回值函数使用规范

### 5.3 数组作为函数参数

- 一维数组传入函数处理数据
- 二维数组传参行列参数规则
- 数组传参本质地址传递特性

### 5.4 变量存储属性分类

- 局部变量作用域、生命周期
- 全局变量作用范围与优缺点
- static 静态局部变量常驻内存特性
- static 静态全局变量文件访问限制
- extern 外部变量跨文件引用调用
- auto、register 存储类别说明符

### 5.5 递归函数

- 递归思想：自身调用自身逻辑
- 递归必备终止条件防止死递归
- 阶乘、斐波那契数列递归实现
- 递归遍历、简单回溯基础案例
- 递归栈溢出原因与优化思路

## 第六章指针核心重难点

### 6.1 指针基础原理

- 内存地址、内存单元、指针变量
- 取地址运算符 &、解引用运算符 \*
- 指针变量定义、赋值、取值操作
- NULL 空指针含义与安全使用

### 6.2 指针与数组结合

- 数组名本质首元素地址
- 指针加减运算遍历数组元素
- 一维数组指针遍历写法
- 二维数组指针访问行列数据
- 数组指针、指针数组概念区分辨析

### 6.3 字符指针与字符串

- 字符指针表示字符串常量
- 指针方式改写字符串库函数
- 字符串常量区不可修改特性
- 字符数组与字符指针本质差异

### 6.4 多级指针

- 二级指针定义、赋值、取值
- 二级指针指向一维、二维数据
- 多级指针适用复杂参数场景

### 6.5 函数指针

- 指针作为函数参数传递地址
- 函数指针定义格式与调用方法
- 简单回调函数基础理解

## 第七章自定义复合数据类型

### 7.1 结构体

- 结构体自定义数据类型定义
- 结构体变量多种初始化方式
- 结构体成员点运算符访问赋值
- 结构体嵌套成员定义与访问
- 结构体数组批量存储同类对象
- 结构体指针箭头访问成员变量
- 结构体内存对齐规则与大小计算

### 7.2 共用体 union

- 共用体共享同一块内存特点
- 共用体变量定义与成员赋值
- 共用体数据类型转换应用
- 共用体与结构体存储区别对比

### 7.3 枚举类型 enum

- 枚举定义固定有限状态选项
- 枚举默认数值、自定义赋值
- 枚举用于状态、星期、选项标识

### 7.4 类型重命名 typedef

- 基础数据类型别名简化
- 结构体、指针类型别名简写
- typedef 与 #define 宏定义本质区别

## 第八章动态内存管理

### 8.1 程序内存四区模型

- 栈区存储局部变量特性
- 堆区手动管理动态内存空间
- 全局区存放全局、静态变量
- 常量区字符串常量只读存储

- 四大区域生命周期与访问权限

## 8.2 动态内存标准函数

- malloc 申请指定大小堆内存
- calloc 申请内存并自动初始化 0
- realloc 对已有内存扩容、缩容
- free 释放堆内存避免空间浪费

## 8.3 动态内存常见错误

- 野指针产生原因与规避方法
- 内存泄漏问题检测与修复
- 动态内存越界访问危害排查
- 重复释放、空指针释放错误

## 8.4 动态内存综合应用

- 动态一维数组灵活开辟空间
- 动态结构体数组存储数据

# 第九章 文件读写操作

## 9.1 文件基础概念

- 文本文件、二进制文件存储区别
- 文件指针 FILE\* 作用与文件流
- 文件路径相对路径、绝对路径写法

## 9.2 文件打开关闭模式

- r 只读、w 只写、a 追加写入模式
- r+ 读写、w+ 新建读写、a+ 追加读写
- fopen 打开文件、fclose 关闭文件
- 文件打开失败判断与异常处理

## 9.3 文本文件读写

- fgetc、fputc 单个字符读写文件
- fgets、fputs 整行文本读取写入

- fprintf 格式化写入文件数据
- fscanf 格式化读取文件内容

## 9.4 二进制文件读写

- fread 批量读取二进制块数据
- fwrite 批量写入二进制数据
- 结构体数据二进制持久化存储

## 9.5 文件指针定位

- fseek 移动文件指针指定偏移位置
- rewind 重置文件指针到开头
- ftell 获取当前指针偏移位置

# 第十章预处理指令

## 10.1 头文件包含指令

- #include<> 系统库头文件引入
- #include"" 自定义文件包含
- 头文件重复包含问题与防护

## 10.2 宏定义指令

- 无参宏定义常量替换
- 带参宏定义简易函数替换
- 宏定义换行、运算符注意事项

## 10.3 条件编译指令

- #ifdef、#ifndef 条件判断编译
- #else、#endif 分支编译控制
- 不同平台代码兼容适配用法

## 10.4 其他预处理指令

- #error 编译报错提示指令
- #pragma 编译参数设置指令

## 第十一章基础算法实现

### 11.1 排序算法

- 冒泡排序原理、代码、优化版本
- 选择排序思路与手写实现
- 插入排序逻辑与适用场景

### 11.2 查找算法

- 顺序查找无序数组遍历检索
- 二分查找有序数组高效查找

### 11.3 常用基础算法案例

- 斐波那契数列迭代递归写法
- 素数判断、水仙花数统计
- 最大公约数、最小公倍数求解

## 第十二章综合项目实战

### 12.1 简易控制台计算器

- 加减乘除四则运算实现
- 循环连续计算、异常输入拦截

### 12.2 学生信息管理系统

- 结构体存储学号姓名成绩
- 信息新增、查询、修改、删除
- 成绩排序、总分平均分统计

### 12.3 本地文件通讯录系统

- 联系人姓名电话存储
- 信息读写保存到本地文件
- 联系人查找、新增、删除功能

## 第十三章 程序调试与排错

### 13.1 错误类型区分

- 语法错误编译直接报错
- 运行错误程序异常终止
- 逻辑错误结果不符合预期

### 13.2 常见经典报错总结

- 下标越界、空指针访问错误
- 内存泄漏、重复释放内存
- 函数声明定义参数不匹配
- 字符串结束符缺失异常

### 13.3 基础调试方法

- printf 打印中间变量排查逻辑
- 断点调试、分步执行查看数据
- 边界值测试验证程序稳定性

Math

数学

宇宙最美的语言

2026 年 6 月 5 日



## 11 Math

# Math Day

## 数学一道

你如同一个吻，封碱了你的嘴

2026 年 6 月 5 日

**11.1 math day**

## 26 May

Our patience will achieve more than our force.

我们的耐心将比武力取得更多成就。

— Edmund Burke(埃德蒙·伯克)

TODO: 2026-05-26

# Math CUM

## 数学建模

你如同一个吻，封碱了你的嘴

2026 年 6 月 5 日

**11.2 cum**

这是数学建模。

Other

数学

你如同一个吻，封碱了你的嘴

2026 年 6 月 5 日

### 11.3 symbols



Symbols

$\int \cdots \int \oint$

$\oint$  路径积分  
需要导入 esin 包。  
 $\oint$   
 $\oint$

# PHSICS

## 物理

你如同一个吻，封碱了你的嘴

2026 年 6 月 5 日

## 12 Physics

# Physics Day

## 物理

你如同一个吻，封碱了你的嘴

2026 年 6 月 5 日

## **12.1 physics day**

## 26 May

Our patience will achieve more than our force.

我们的耐心将比武力取得更多成就。

— Edmund Burke(埃德蒙·伯克)

TODO: 2026-05-26

## 12.2 draft

### code

```
1 def func(x):
2     return x ** 2 + 1
3
4 if __name__ == "__main__":
5     print(func(5))
```

```
1 #include <stdio.h>
2 int main() {
3     printf("Dear Sophiw. 你好呀! ");
4
5     return 0;
6 }
```

引用头文件

printf 输出函数

注意格式化字符串和数据匹配

### note

The original todo note with changed colours. Here's another line.

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Ut purus elit, vestibulum ut, placerat ac, adipiscing vitae, felis. Curabitur dictum gravida mauris. Nam arcu libero, nonummy eget, consectetur id, vulputate a, magna. Donec vehicula augue eu neque. Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas. Mauris ut leo. Cras viverra metus rhoncus sem. Nulla et lectus vestibulum urna fringilla ultrices. Phasellus eu tellus sit amet tortor gravida placerat. Integer sapien est, iaculis in, pretium quis, viverra ac, nunc. Praesent eget sem vel leo ultrices bibendum. Aenean faucibus. Morbi dolor nulla, malesuada eu, pulvinar at, mollis ac, nulla. Curabitur auctor semper nulla. Donec varius orci eget risus. Duis nibh mi, congue eu, accumsan eleifend, sagittis quis, diam. Duis eget orci sit amet orci dignissim rutrum. Lorem ipsum dolor sit amet, consectetur adipiscing elit. Ut purus elit, vestibulum ut, placerat ac, adipiscing vitae, felis. Curabitur dictum gravida mauris. Nam arcu libero, nonummy eget, consectetur id, vulputate a, magna. Donec vehicula augue eu neque. Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas. Mauris ut leo. Cras viverra metus rhoncus sem. Nulla et lectus vestibulum urna fringilla ultrices. Phasellus eu tellus sit amet tortor gravida placerat. Integer sapien est, iaculis in, pretium quis, viverra ac, nunc. Praesent eget sem vel leo ultrices bibendum. Aenean faucibus. Morbi dolor nulla, malesuada eu, pulvinar at, mollis ac,

Is this correct?

I'm unsure about also!

nulla. Curabitur auctor semper nulla. Donec varius orci eget risus. Duis nibh mi, congue eu, accumsan eleifend, sagittis quis, diam. Duis eget orci sit amet orci dignissim rutrum. Sed feugiat. Cum sociis natoque penatibus et magnis dis parturient montes, nascetur ridiculus mus. Ut pellentesque augue sed urna. Vestibulum diam eros, fringilla et, consectetur eu, nonummy id, sapien. Nullam at lectus. In sagittis ultrices mauris. Curabitur malesuada erat sit amet massa. Fusce blandit. Aliquam erat volutpat. Aliquam euismod. Aenean vel lectus. Nunc imperdiet justo nec dolor. \_\_\_\_\_ Sed feugiat. Cum sociis natoque penatibus et magnis dis parturient montes, nascetur ridiculus mus. Ut pellentesque augue sed urna. Vestibulum diam eros, fringilla et, consectetur eu, nonummy id, sapien. Nullam at lectus. In sagittis ultrices mauris. Curabitur malesuada erat sit amet massa. Fusce blandit. Aliquam erat volutpat. Aliquam euismod. Aenean vel lectus. Nunc imperdiet justo nec dolor. \_\_\_\_\_ Sed feugiat. Cum sociis natoque penatibus et magnis dis parturient montes, nascetur ridiculus mus. Ut pellentesque augue sed urna. Vestibulum diam eros, fringilla et, consectetur eu, nonummy id, sapien. Nullam at lectus. In sagittis ultrices mauris. Curabitur malesuada erat sit amet massa. Fusce blandit. Aliquam erat volutpat. Aliquam euismod. Aenean vel lectus. Nunc imperdiet justo nec dolor.

Change this!

This can help me in chapter seven!

This really needs to be improved! What was I thinking?!

The following section needs to be rewritten!

Sed feugiat. Cum sociis natoque penatibus et magnis dis parturient montes, nascetur ridiculus mus. Ut pellentesque augue sed urna. Vestibulum diam eros, fringilla et, consectetur eu, nonummy id, sapien. Nullam at lectus. In sagittis ultrices mauris. Curabitur malesuada erat sit amet massa. Fusce blandit. Aliquam erat volutpat. Aliquam euismod. Aenean vel lectus. Nunc imperdiet justo nec dolor.

## picture

这是图片





图 1: 这是图片的标题

创建自己的作品集，照片，视频，积累

**picture**



图 2: 三张图片水平并列

picture



(a) 子图 1



(b) 子图 2



(c) 子图 3

图 3: 带子标题的多图示例

picture



(a) 子图 1



(b) 子图 2



(c) 子图 3



(d) 子图 4

图 4: 2×2 网格多图

图 3a 是第一张图，图 3 是整体图组  
[点击播放音频](#)

[点击播放视频](#)

**动画**

## 结语